



TEMA II

DESARROLLO DE APLICACIONES BACK EN PHP

*"Vive como si fueses a morir mañana,
aprende como si fueses a vivir siempre"*

Mahatma Gandhi.

5. PROGRAMACIÓN CLIENTE/SERVIDOR CON HTTP

Dada su naturaleza de lenguaje de lado servidor, PHP es capaz de darnos acceso a toda una serie de variables que nos informan sobre nuestro servidor y sobre el cliente. La información de estas variables es atribuida por el servidor y en ningún caso nos es posible modificar sus valores directamente mediante el script.

Existen multitud de variables de este tipo, algunas sin utilidad aparente y otras realmente interesantes y con una aplicación directa para nuestro sitio web

Estas variables de tipo array que mantienen información del sistema, son llamadas **superglobales** porque se definen automáticamente en un ámbito global, es decir están disponible desde cualquier punto de tu aplicación.

No todas estas variables están disponibles en la totalidad de servidores o en determinadas versiones de un mismo servidor. Además, algunas de ellas han de ser previamente activadas o definidas por medio de algún "acontecimiento". Así, por ejemplo, la variable \$HTTP_REFERER no estará definida a menos que el internauta acceda al script a partir de un enlace desde otra página.

\$_SERVER

Variables definidas por el servidor web ó directamente relacionadas con el entorno en donde el script se está ejecutando.

Clave	Descripción
HTTP_USER_AGENT	Nos informa principalmente sobre el sistema operativo y tipo y versión de navegador utilizado por el internauta. Su principal utilidad radica en que, a partir de esta información, podemos redireccionar nuestros usuarios hacia páginas optimizadas para su navegador o realizar cualquier otro tipo de acción en el contexto de un navegador determinado.
HTTP_ACCEPT_LANGUAGE	Nos devuelve la o las abreviaciones de la lengua considerada como principal por el navegador. Esta lengua o lenguas principales pueden ser elegidas en el menú de opciones del navegador. Esta variable resulta también extremadamente útil para enviar al internauta a las páginas escritas en su lengua, si es que existen.
HTTP_REFERER	Nos indica la URL desde la cual el internauta ha tenido acceso a la página. Muy interesante para generar botones de "Atrás" dinámicos o para crear nuestros propios sistemas de estadísticas de visitas.
PHP_SELF	Nos devuelve una cadena con la URL del script que está siendo ejecutado. Muy interesante para crear botones para recargar la página.
PHP_AUTH_USER	Almacena la variable usuario cuando se efectúa la entrada a páginas de acceso restringido. Combinado con \$PHP_AUTH_PW resulta ideal para controlar el acceso a las páginas internas del sitio.
PHP_AUTH_PW	Almacena la variable password cuando se efectúa la entrada a páginas de acceso restringido. Combinado con \$PHP_AUTH_USER resulta ideal para controlar el acceso a las páginas internas del sitio.
REMOTE_ADDR	Muestra la dirección IP del visitante.
DOCUMENT_ROOT	Nos devuelve el path físico en el que se encuentra alojada la página en el servidor.
PHPSESSID	Guarda el identificador de sesión del usuario. Veremos más adelante en qué consisten las sesiones.

\$_GET

Variables proporcionadas al script por medio de HTTP GET.

```
Localhost/ejemparam.php?numero=3&nombre="aaa"

$n=$_GET['nombre'];
$num=$_GET['numero'];
```

\$_POST

Variables proporcionadas al script por medio de HTTP POST.

```
<form method=POST>
<input type="text" name="usuario">
</form>

$_POST['usuario'].
```

\$_REQUEST

Variables proporcionadas al script por medio de cualquier mecanismo de entrada del usuario y por lo tanto no se puede confiar en ellas. La presencia y el orden en que aparecen las variables en esta matriz es definido por la directiva de configuración `variables_order`

Contiene todos los datos recibidos mediante los métodos Get, Post y Cookies. Si el script recibe un dato con el mismo nombre a través de varios de estos métodos, prevalecerá el del que ocupe la última posición (más a la derecha) en la directiva `request_order` del archivo `php.ini`. Por ejemplo, en WAMP se utiliza la directiva `request_order = "GP"`, que da prioridad a los datos recibidos a través del método POST y no incluye en `$_REQUEST` los datos recibidos a través de cookies (por motivos de seguridad).

\$_COOKIE

Variables proporcionadas al script por medio de HTTP cookies. Contiene los datos recibidos del cliente a través de cookies. Esos datos deben haber sido previamente almacenados en la cookie del cliente por el mismo servidor (las cookies sólo pueden ser leídas por el propio servidor que las creó).

\$_FILES

Variables proporcionadas al script por medio de la subida de ficheros via HTTP. Contiene los archivos enviados a través de elementos input type='file' de un formulario.

\$_ENV

Variables proporcionadas al script por medio del entorno. Contiene datos establecidos por el sistema operativo. Al igual que ocurre con \$_SERVER no todos los sistemas establecen los mismos datos. Por defecto esta variable está vacía en algunos entornos, por ejemplo WAMP, porque en la directiva variables_order del archivo php.ini no se incluye la letra E de Environment.

\$_SESSION

Variables registradas en la sesión del script. A diferencia de las cookies, las variables de sesión no se envían al cliente.

Ámbito de las variables

En PHP disponemos de los siguientes ámbitos:

- **Superglobal.** Son variables predefinidas, su valor está definido por el intérprete de PHP, no podemos crearlas nosotros, se pueden utilizar en cualquier posición del código. Todas las variables superglobales son arrays.
- **Global.** cuando utilicemos una variable fuera de una función, será reconocida como tal fuera de la misma (esto incluye a los archivos externos que pudiéramos incrustar en el script mediante instrucciones como include o require). Para utilizarlas dentro de las funciones tenemos que indicar expresamente que queremos referirnos a la variable global, utilizando uno de estos dos métodos:

- Refiriéndonos a la variable global a través de la superglobal \$GLOBALS
- Declarándola como global dentro de la función mediante la instrucción global.

global \$nombre

- **Local.** Una variable utilizada dentro de una función sólo es reconocida dentro de ella, su valor se pierde cuando termina la ejecución de la función.
- **Estático.** Una variable estática dentro de una función es local a la misma, pero no pierde su valor cuando termina la ejecución, lo conserva para la siguiente ejecución.

static \$contador=1;

se inicializa la variable, esta instrucción solo se ejecutará la primera vez que se llame a la función. En el resto de ejecuciones la variable mantiene el valor de la anterior, pero no lo mantendrá al cargar de nuevo la página.

5.2 Trabajando con formularios HTML

Un formulario HTML se define con el elemento `<form>`, cuyos atributos principales son:

- `action`: archivo html, php,...dónde se envían los datos recogidos en el formulario
- `method`: Para indicar el método de envío de datos: POST o GET

Dentro del formulario,

- El atributo `name` de los controles es el que envía el formulario junto con el dato recogido por el campo.
- Los botones de opción `<input type='radio'>` tendrán el mismo `name`. El formulario pasa el `value` del seleccionado
- Los controles que pueden almacenar varios valores (las casillas de verificación) tendrán como `name` un array(`[]`).

```
<input type='checkbox' name='aficiones[]'
value='musica' /><br/>
<input type='checkbox' name='aficiones[]'
value='deporte' /><br/>

<select name='aficiones[]' size='5' multiple='multiple'>
  <option value='lectura'>Lectura</option>
  <option value='musica'>Música</option>
  <option value='deporte'>Deporte</option>
  <option
value='informatica'>Informática</option>
  <option
value='electronica'>Electrónica</option>
</select>
```

Estos datos se recogen en PHP con las variables `$_POST` o `$_GET`, según el método de envío elegido.

El tamaño de los datos que se pueden enviar a través de peticiones GET suele ser menor que el del método POST. La limitación para el método GET puede encontrarse en el propio navegador del cliente, que

trunque la URL a 255 caracteres, o en el lado del servidor Apache, donde el tamaño máximo está controlado por la **directiva LimitRequestLine** del **httpd.conf**.

En el caso del método POST, la **directiva post_max_size** del **php.ini** determina el tamaño máximo de los datos que pueden enviarse. Pero esta directiva está supeditada al valor establecido en la directiva **LimitRequestBody** del **httpd.conf** de Apache.

Un error frecuente es dar por hecho que vamos a recibir en nuestro script PHP todas las variables que esperamos y empezar a operar con ellas directamente. Esto producirá un error si alguna de esas variables no estuviera definida.

Para evitarlo, es conveniente recoger los valores de `$_POST` y `$_GET` en variables globales del sistema, comprobando previamente que realmente están asignadas.

Ejemplo

```
<?php
    if(isset($_GET['usuario'])){
        $usu=$_GET['usuario'];
    }else{
        die('Credenciales incompletas');
    }
?>
```

5.3 Trabajando con varios archivos

Para utilizar en un archivo php código de funciones definidas en otro archivo php nos proporciona dos funciones: ***include*** y ***require***

Sirven para insertar código procedente de un archivo dentro de otro. La sintaxis de ambas es la misma:

```
include(ruta_de_archivo);
require(ruta_de_archivo);
```

- **include**: si no encuentra el archivo emite un aviso pero no detiene la ejecución del script;
- **require**: produce un error y detiene la ejecución si el archivo no se encuentra

La ruta puede expresarse:

- **Explícitamente:** De forma absoluta o relativa (usando `.` para el directorio actual y `..` para el directorio padre).
- **Implícitamente:** Si escribimos simplemente el nombre de un archivo (sin ruta), PHP intentará localizarlo buscando en el siguiente orden
 - En la carpeta especificada en la directiva `include_path` del `php.ini`, `.htaccess` o `ini_set`
 - En el propio directorio donde se encuentra el archivo php que ha ordenado la inclusión.

En cuanto lo encuentra en uno de estos destinos no sigue buscando en los demás

Al ejecutar una instrucción `include` o `require`, se detiene el intérprete PHP, se abre el archivo que se quiere incluir y se incrusta su contenido en el archivo original. El archivo incluido podrá ser HTML o php.

El código incluido adquiere el ámbito de la posición en la que es incrustado. Por ejemplo:

Ejemplo

```
<?php
    ini_set('include_path','c:\\wamp\\www\\');
    define('INCLUIDO',TRUE);
    require('codigo_tabla_multiplicar.php');
    tabla_de_multiplicar(9);
?>
```

`tabla_de_multiplicar.php`

```
<?php
    if(!defined('INCLUIDO'))
        die('No autorizado');

    for($i=1;$i<=10;$i++)
        echo $numero.' x '.$i.' = '. ($numero*$i). '<br />';

?>
```

Al utilizar archivos incluidos es frecuente definir en el archivo que va a realizar la inclusión una constante, y luego verificar al principio del archivo si esa constante está definida. Esto evita que se puedan ejecutar directamente los archivos incluidos sin incluirlos en ningún otro archivo (lo que podría suponer una amenaza de seguridad).

Se puede incluir un archivo o una URL, para hacer esto último es necesario activar una directiva en el `php.ini` `allow_url_include`

```
include(http://localhost/ejercicios_2/dos/verNota.php?posicion=$res);
```

Existe otra pareja de funciones **`include_once`** y **`require_once`**. La diferencia con las anteriores es que éstas últimas verifican si el archivo ya ha sido incluido, si es así no lo incluyen de nuevo.

5.4 Redirigiendo a otra página

Para redirigir al cliente a otra dirección utilizamos la función `header`

```
header('Location: página.php');
```

Se pueden pasar parámetros

```
header('Location: errores.php?tipo=1');
```

La cabecera `Refresh`, nos permite establecer un plazo en segundos antes de redirigir al navegador a otra dirección. Por ejemplo:

```
header('Refresh: 5; url=http://www.google.es');  
echo 'Lo que busca no existe, le redirigiremos a Google en 5 segundos'
```

Realmente sirve para enviar cabeceras al cliente, lo veremos con más profundidad cuándo veamos microservicios. Esta función recibe un único argumento, el texto de la cabecera que deseamos enviar.

La función `header` da problemas si se encuentra espacios en blanco después o antes del inicio o fin del código PHP. Para evitar problemas con el envío de las cabeceras utilizaremos un `buffer`

PHP dispone de funciones para controlar en la salida de datos hacia el cliente. Se puede almacenar la salida en un `buffer`, para enviarla al cliente cuando se desee. Por regla general PHP va enviando el código html según va procesando la página. Esta configuración puede cambiarse. PHP dispone de las "funciones de control de salida" `:ob_start()` y `ob_end_flush()`.

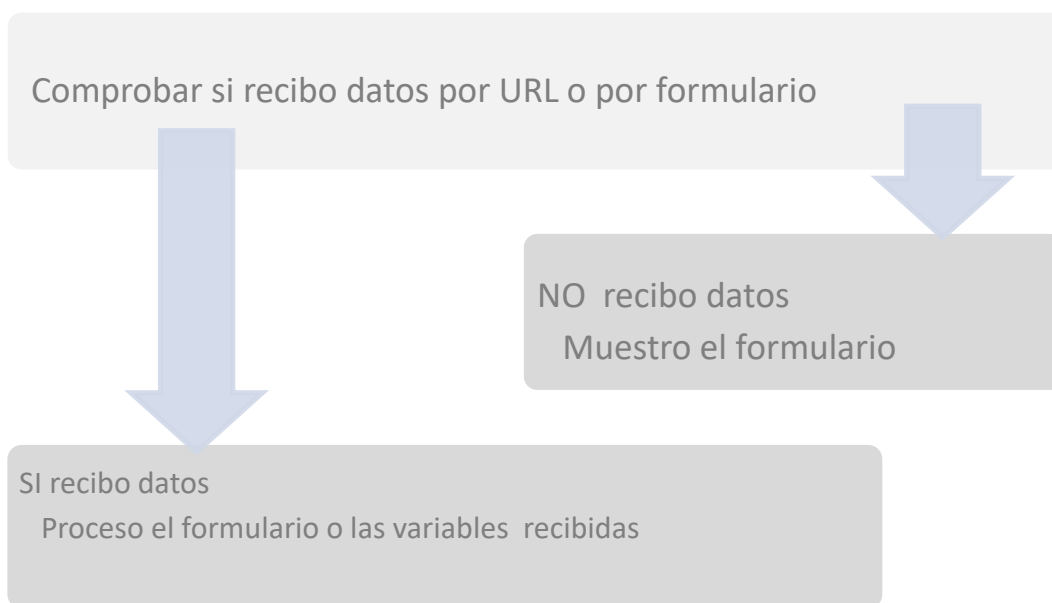
La función `ob_start()` sirve para indicarle a PHP que debe empezar a guardar la salida en un buffer interno, en vez de enviarla al cliente. Aunque se escriba código HTML con `echo` o directamente fuera del código PHP, no se enviará al navegador hasta que se ordene explícitamente o hasta que se acabe el procesamiento de toda la página

La función `ob_end_flush()` sirve para indicar a PHP que realice el volcado de todo el buffer en la salida, con lo que se enviará al cliente que ha solicitado la página.

5.5 AutoLlamada de páginas

Se pueden enviar y recibir datos de un formulario con una única página. A este efecto podemos llamarle "**autollamada de páginas**", también se le suele llamar como "**Formularios reentrantes**".

En ambos casos, para formularios o envío de datos por la URL, se debe seguir el siguiente esquema:



Ejemplo

1. Si no se reciben datos desde el formulario, se muestra éste. Si se reciben se procesan

```
<html> <head> <title>Me llamo a mi mismo...</title></head>
<body>
<?php
if (!isset($_POST['enviar'])){
?><form action="auto-llamada.php" method="post">
    Nombre:<input type="text" name="nombre" size="30"><br>
    Empresa:<input type="text" name="empresa" size="30"><br>
    Telefono:<input type="text" name="telefono" size=14 value="+34"><br>
    <input type="submit" name="enviar" value="Enviar">
</form>
<?php
}else{
    echo "<br>Su nombre: " . $_POST["nombre"];
    echo "<br>Su empresa: " . $_POST["empresa"];
    echo "<br>Su Teléfono: " . $_POST["telefono"];
}
?>
</body>
</html>
```

2. Si no se reciben datos por la URL se muestran los enlaces para ver cada una de las tablas. Si se reciben datos, se muestra la tabla de multiplicar del número que se está recibiendo en la URL.

```
<?php
if (!$_GET['tabla']){
    for ($i=1;$i<=10;$i++){
        echo "<br><a href='ver_tabla.php?tabla=$i'>Ver la tabladel$ i</a>\n";    }
    } else {
        $tabla=$_GET["tabla"];
    ?>
    <table align=center border=1 cellpadding="1">
<?php
    for ($i=0;$i<=10;$i++){
        echo "<tr><td>$tabla X $i</td><td>=</td><td>" . $tabla * $i . "</td></tr>\n";
    } ?>
</table><? php } ?>
```